

Client WOE Entry

Full-Stack Assignment — Solution Documentation

ASP.NET Core 8 Web API • EF Core • SQL Server • HTML/CSS/JS Frontend

Prepared by: Vishnu | Bangalore

Submission package: source code (backend + frontend), SQL script, README, and this documentation covering architecture, database design, and API reference.

1. Objective

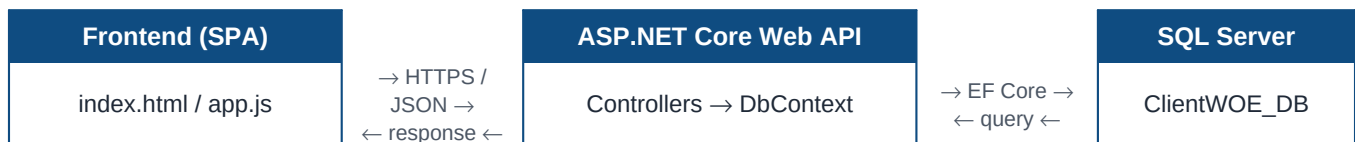
Implement a simplified Client Work Order Entry (WOE) module: capture client and patient details, search and select laboratory tests, review the total amount, and submit the order through an API that generates a unique WOE/Order Number and persists the order and its test line items in SQL Server.

Requirements Covered

- Client selection, patient details capture (name, ID, age, gender, mobile)
- Test catalog display with search, multi-select, quantity, running total
- WOE submission via API with a generated unique order number
- Saved order + line items persisted in SQL Server, shown back to the user on success
- RESTful ASP.NET Core Web API with model validation, proper HTTP status codes, global exception handling, EF Core (Code-First), and Swagger
- Normalized SQL Server schema for Client, Patient, TestMaster, WorkOrder, WorkOrderTestDetail with FK relationships
- Frontend: client/patient form, test search & selection grid, total calculation, submit flow, basic validation and error handling

2. Architecture Overview

A conventional 3-tier layout: a static HTML/CSS/JS frontend calls a stateless ASP.NET Core Web API over JSON/HTTPS; the API uses EF Core (Code-First) against SQL Server. No build tooling is required for the frontend, keeping the whole stack easy to run and review.



Clients / Patients / TestMaster / WorkOrders APIs | Tables: Client, Patient, TestMaster, WorkOrder, WorkOrderTestDetail

Backend Layers

- **Controllers** — one per resource (Clients, Patients, TestMaster, WorkOrders); WorkOrdersController owns the core submission workflow.
- **Models** — EF Core entities with DataAnnotations for validation.
- **DTOs** — request/response contracts for WOE submission, decoupled from the entities so the API shape stays stable even if the schema evolves.
- **Data** — WoeDbContext: fluent-API relationships, unique indexes, and seed data (clients + test catalog) via HasData().

3. Database Design

Five tables, normalized, with explicit foreign keys and unique constraints on business keys.

Table	Key Columns	Purpose
Client	ClientId (PK), ClientCode (unique)	Referring clients (hospitals, walk-in, etc.)

Table	Key Columns	Purpose
Patient	PatientId (PK), PatientCode (unique), ClientId (FK)	Patients, each linked to one Client
TestMaster	TestId (PK), TestCode (unique), Rate	Catalog of available lab tests and prices
WorkOrder	WorkOrderId (PK), WoeNumber (unique), ClientId (FK), PatientId (FK), TotalAmount	WOE header — one row per submitted order
WorkOrderTestDetail	WorkOrderTestDetailId (PK), WorkOrderId (FK, cascade), TestId (FK), Qty, Rate, Amount	Line items — one row per test on an order

Relationships

- Client (1) → Patient (many)
- Client (1) → WorkOrder (many)
- Patient (1) → WorkOrder (many)
- WorkOrder (1) → WorkOrderTestDetail (many), cascade delete
- TestMaster (1) → WorkOrderTestDetail (many)

The rate is copied onto `WorkOrderTestDetail.Rate` at order time, so a later change to `TestMaster.Rate` never alters historical orders — the same pattern used for invoice line-item integrity in production lab systems.

Provisioning

- **Option A (recommended):** EF Core migrations — the API also auto-applies pending migrations on startup via `db.Database.Migrate()`.
- **Option B:** run `database/schema.sql` directly — a standalone SQL Server script that creates the same tables, indexes, and seed data.

4. API Reference

Full interactive documentation is served by Swagger at `/swagger` once the API is running. Summary of endpoints:

Method	Route	Description
GET	<code>/api/clients</code>	List active clients
GET	<code>/api/clients/{id}</code>	Get a single client
POST	<code>/api/clients</code>	Create a client
GET	<code>/api/patients?search=&clientId;=</code>	Search / list patients
GET	<code>/api/patients/{id}</code>	Get a single patient
GET	<code>/api/testmaster?search=</code>	List / search the test catalog
GET	<code>/api/testmaster/{id}</code>	Get a single test
GET	<code>/api/workorders?clientId=&from;=&to;=</code>	List work orders, optionally filtered
GET	<code>/api/workorders/{id}</code>	Get a saved work order with its line items
POST	<code>/api/workorders</code>	Submit a WOE — creates patient if new, generates WoeNumber, saves order + line items in one transaction

Sample Request — POST `/api/workorders`

```
{
  "clientId": 2,
  "patientName": "Ravi Kumar",
  "age": 34,
  "gender": 0,
  "mobileNumber": "9880012345",
  "tests": [
    { "testId": 1, "quantity": 1 },
    { "testId": 3, "quantity": 1 }
  ]
}
```

Sample Response — 201 Created

```
{
  "workOrderId": 1,
  "woeNumber": "WOE-20260827-0001",
  "orderDate": "2026-08-27T10:15:00Z",
  "clientName": "Apollo Diagnostics",
  "patientCode": "PAT-000001",
  "patientName": "Ravi Kumar",
  "age": 34,
  "gender": "Male",
  "mobileNumber": "9880012345",
  "totalAmount": 1050.00,
  "status": "Saved",
  "tests": [
    { "testId": 1, "testName": "Complete Blood Count",
      "rate": 350.00, "quantity": 1, "amount": 350.00 },
    { "testId": 3, "testName": "Liver Function Test",
      "rate": 700.00, "quantity": 1, "amount": 700.00 }
  ]
}
```

HTTP Status Codes Used

Code	When
200 OK	Successful GET requests
201 Created	WorkOrder / Client created — includes Location header
400 Bad Request	Model validation failure, missing/invalid test or patient fields
404 Not Found	Requested Client / Patient / Test / WorkOrder id doesn't exist
409 Conflict	Duplicate client code on create
500 Internal Server Error	Unhandled exception — caught by global handler, logged, generic message returned

5. Key Design Decisions

WOE Number Generation

Format WOE-yyyyyMMdd-#####, sequence resets daily. Generated inside the same database transaction as the order save (patient creation, order header, and line items all commit or roll back together), with a defensive re-check loop to guard against a race between two concurrent submissions on the same day.

Validation

- DataAnnotations on entities/DTOs (required fields, ranges, max lengths, phone format).
- Explicit business checks in WorkOrdersController: client must exist, patient must exist or have all required new-patient fields, at least one test line, each TestId must resolve to an active test.
- Invalid requests return 400 with either ModelState field errors or a structured message.

Exception Handling

- Global UseExceptionHandler middleware returns a consistent JSON error shape and logs the exception server-side without leaking internals to the client.
- The WOE save wraps the multi-table insert (patient + order + line items) in an explicit EF Core transaction with rollback on any failure.

Frontend

- Framework-free HTML/CSS/JS — no build step, fast to run and review.
- Live test search (debounced), existing-patient lookup with autocomplete, editable quantities with an auto-recalculating total, and inline validation before submit.
- On success, the saved WOE (order number, patient, line items, total) is rendered back to the user, matching requirement 3 ('display the saved WOE details after successful submission').

Deliberately Simplified

This is a scoped take-home assignment, not a production system: no authentication/authorization, no soft-delete/audit trail beyond IsActive/Status flags, and pagination is a simple row cap rather than full paging. These are natural next steps for a production rollout.

6. Setup Instructions

Prerequisites

- .NET 8 SDK
- SQL Server (LocalDB, Developer Edition, Docker, or Azure SQL)
- Any modern browser (no npm/build tooling needed for the frontend)

Backend

```
cd backend/ClientWOE.API
dotnet restore
```

```
# Point ConnectionStrings:DefaultConnection in appsettings.json at your SQL Server, e.g.:
# "Server=(localdb)\MSSQLLocalDB;Database=ClientWOE_DB;Trusted_Connection=True;"
dotnet tool install --global dotnet-ef # first time only
```

```
dotnet ef migrations add InitialCreate
dotnet ef database update

dotnet run
# API:      https://localhost:7080
# Swagger:  https://localhost:7080/swagger
```

Frontend

```
# confirm API_BASE in frontend/app.js matches your API URL, then:
cd frontend
python3 -m http.server 8080
# open http://localhost:8080
```

Alternative: run the raw SQL script

Create an empty `ClientW0E_DB` database and run `database/schema.sql` directly — it creates the same tables, indexes, and seed data as the EF Core migration.

7. Project Structure

```
ClientWOE/
├── backend/
│   ├── ClientWOE.API/
│   │   ├── Controllers/      Clients, Patients, TestMaster, WorkOrders
│   │   ├── Models/          EF Core entities
│   │   ├── DTOs/             Request/response contracts for WOE submission
│   │   ├── Data/             WoeDbContext (EF Core, seed data)
│   │   ├── Program.cs        Startup, Swagger, CORS, global exception handling
│   │   ├── appsettings.json
│   │   └── ClientWOE.API.csproj
│   └── database/
│       ├── schema.sql        Standalone SQL Server script (tables + seed data)
│       └── frontend/
│           ├── index.html
│           ├── app.js
│           └── styles.css
└── README.md
```

8. Demonstration Checklist

Once both the API and frontend are running locally, the following flow demonstrates every requirement:

- Open `/swagger` — confirm all endpoints and try `POST /api/workorders` from the Swagger UI directly.
- Open the frontend — pick a client, enter patient details (or search an existing patient).
- Search the test catalog, click tests to add them, adjust quantities — confirm the total updates live.
- Click **Save Work Order** — confirm a unique WOE number is returned and the saved order (patient, line items, total) renders on screen.
- Re-run the submission — confirm the sequence in the WOE number increments and stays unique.
- Try submitting with no tests selected, or an invalid mobile number — confirm inline validation blocks the call before it reaches the API, and that a malformed direct API call returns 400 with a clear message.

Screenshots are best captured live against your own SQL Server instance and are not included in this document — see the README's Screenshots/Demo section for the exact steps to capture them.